

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении практической работы №5
по дисциплине «Функциональное программирование»
Вариант №42.

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Моторин Д.Е.

«_____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Постановка задачи	4
2 Разработка системы обработки заказов	5
2.1 Типы данных	5
2.2 Логирование	6
2.3 Проверка заказа	7
2.4 Главная функция	7
3 Проверка корректности	9
Выводы	10
Список использованных источников	11
Приложение 1 «Исходный код программы»	12

Введение

Интернет-магазин - это онлайн-платформа, где покупатели могут выбирать товары, формировать заказы и оплачивать их онлайн. В зависимости от выбора покупателя товар может доставляться как в пункт выдачи, так и курьером.

В каждый заказ может входить:

1. Уникальный номер заказа.
2. Имя покупателя.
3. Список товаров.
4. Статус заказа.
5. Дата создания.

Для корректной работы интернет-магазина нужно проверять заказ до подтверждения и сообщать покупателю о возникающих проблемах. Часто могут возникать ошибки:

1. Товара нет в наличии.
2. Неверно указаны данные покупателя.
3. Пустой заказ (не выбрано ни одного товара).
4. Проблемы с оплатой.

При обнаружении ошибок система должна их обрабатывать. Логи могут быть удобны для отслеживания этапов обработки заказа и быстрого нахождения проблем.

1 Постановка задачи

Цель: реализация системы обработки заказов с логированием и обработкой ошибок.

Задачи:

1. Определить типы данных `Order`, `Inventory` и `OrderError`.
2. Написать функцию логирования `logMessage`.
3. Написать функцию проверки заказа `validateOrder`.

2 Разработка системы обработки заказов

2.1 Типы данных

Тип `Item` обозначает тип товара в магазине. Тип `OrderId` предназначен для уникального идентификатора заказа. Тип `CustomerName` представляет имя покупателя.

Листинг 1. Типы `Item`, `OrderId`, `CustomerName`

```
1 type Item = String
2 type OrderId = Int
3 type CustomerName = String
```

Тип `Order` определен как тип-произведение и представляет информацию о заказе. Он содержит уникальный идентификатор заказа, имя покупателя и список товаров. Тип получает возможность сравнения и вывода на печать.

Листинг 2. Тип `Order`

```
1 data Order = Order{
2   orderId :: OrderId
3   , customerName :: CustomerName
4   , items :: [Item]
5 } deriving (Show, Eq)
```

Тип `Inventory` представляет текущий склад магазина и содержит список доступных товаров.

Листинг 3. Тип `Inventory`

```
1 type Inventory = [Item]
```

Тип `OrderError` определен как тип-сумма, который представляет возможные ошибки при обработке заказа: товара нет в наличии (`ItemNotAvailable`) и пустой список товаров (`EmptyOrder`).

Листинг 4. Тип `OrderError`

```
1 data OrderError
2 = EmptyOrder
3 | ItemNotAvailable Item
4 deriving (Show, Eq)
```

Тип `OrderProcessor` представляет трансформер монад. Он комбинирует монады `Writer` (накопление логов) и `Except` (обработка ошибок).

Листинг 5. Тип `OrderProcessor`

```
1 type OrderProcessor = ExceptT OrderError (Writer [String])
```

Выполнение `OrderProcessor` всегда возвращает лог и значение, которое может быть как ошибкой, так и успешным результатом. При обработке ошибок `ExceptT` дальнейшие вычисления прекращаются. Так как `Writer` является внутренней монадой, лог накапливается в любом случае, даже при возникновении ошибки.

2.2 Логирование

Функция `writeLog` предназначена для записи сообщений в лог с помощью функции `tell` монады `Writer`, которая добавляет строку сообщения в постепенно собирающийся лог.

Листинг 6. Функция `writeLog`

```
1 writeLog :: String -> OrderProcessor ()
2 writeLog message = tell [message]
```

Функция `logMessage` предназначена для логирования информации о начале обработки заказа вида «Processing order # <OrderId> for <CustomerName>, Items: <items>», где `OrderId`, `CustomerName` и `items` заменяются данными из переданного заказа. С помощью `show` число преобразуется в строку, а `intercalate` объединяет список товаров в строку с разделителями в виде запятых. Логирующее сообщение создается с помощью `writeLog`.

Листинг 7. Функция `logMessage`

```
1 logMessage :: Order -> OrderProcessor ()
2 logMessage order = do
3   writeLog $ "Log: Processing order #" ++ show (orderId order) ++ " for
   " ++ customerName order ++ ". Items: " ++ intercalate ", " (items
   order)
```

Функция `logSuccess` предназначена для логирования информации об успешном завершении обработки заказа.

Листинг 8. Функции `logSuccess`, `logError`

```
1 logSuccess :: Order -> OrderProcessor ()
2 logSuccess order =
3   writeLog $ "Log: Order #" ++ show (orderId order) ++ " successfully
   processed!"
4
5 logError :: OrderError -> OrderProcessor ()
6 logError error =
7   writeLog $ "Log: error: " ++ show error
```

Функция `logError` предназначена для логирования ошибок, возникающих при обработке заказа. С помощью `show` выводится конкретный тип ошибки, определенный в типе `OrderError`.

2.3 Проверка заказа

Функция `validateOrder` предназначена для проверки заказа. Сначала она логирует начало обработки заказа с помощью `logMessage`, а затем проверяет успешность заказа. Если список товаров пуст - логирует ошибку `EmptyOrder` и прерывает выполнение с помощью `throwError`.

Листинг 9. Функция `validateOrder`

```
1 validateOrder :: Inventory -> Order -> OrderProcessor Order
2 validateOrder inventory order = do
3   logMessage order
4   when (null (items order)) $ do
5     logError EmptyOrder
6     throwError EmptyOrder
7   let checkItem item =
8     if item `elem` inventory
9     then return ()
10    else do
11      logError (ItemNotAvailable item)
12      throwError (ItemNotAvailable item)
13   mapM_ checkItem (items order)
14   logSuccess order
15   return order
```

Для каждого элемента из списка товаров функция проверяет его наличие на складе, если хотя бы один товар отсутствует, то она логирует ошибку `ItemNotAvailable` и прерывает выполнение. Если все проверки пройдены успешно, функция логирует успешную обработку и возвращает заказ.

2.4 Главная функция

Функция `main` инициализирует тестовый склад, заполняя его значениями, выводит информацию о доступных товарах и запрашивает у пользователя необходимые данные заказа (`orderId`, `customerName` и `items`). Она создает объект заказа `order` и вызывает функцию валидации `validateOrder`.

Листинг 10. Функция `main`

```
1 main :: IO ()
2 main = do
3   let testInventory = ["laptop", "phone", "mouse", "keyboard", "monitor",
4     "headphones"]
5   putStrLn ""
6   putStrLn $ "Available items: " ++ intercalate ", " testInventory
7   orderId <- readOrderId
8   customerName <- readCustomerName
9   itemsList <- readItems
10  let order = Order orderId customerName itemsList
11  let (result, logMessages) = runWriter (runExceptT (validateOrder
12    testInventory order))
```

```
11 mapM_ putStrLn logMessages
12 case result of
13 Left error -> putStrLn $ "Result: failed: " ++ show error
14 Right order -> putStrLn $ "Result: success! Order processed."
```

Для выполнения валидации функция использует `runExceptT` и `runWriter`, которые «распаковывают» трансформер монад в кортеж из результат и списка логов. Все сообщения логов выводятся, а затем в зависимости от результата функция печатает итоговый результат обработки заказа.

3 Проверка корректности

Для тестирования корректности программы создан тестовый склад, в котором есть следующие товары: laptop, phone, mouse, keyboard, monitor, headphones.

На рисунке 1 приведен результат работы программы с корректным заказом.

```
Available items: laptop, phone, mouse, keyboard, monitor, headphones
Enter order ID: 3
Enter customer name: dd
Enter items (comma-separated): laptop
Log: Processing order #3 for dd. Items: laptop
Log: Order #3 successfully processed!
Result: success! Order processed.
```

Рис. 1. Успешный корректный заказ

На рисунке 2 - 3 приведены результаты работы программы в случаях возникновения ошибки: товара нет в наличии и пустой список товаров при заказе.

```
Available items: laptop, phone, mouse, keyboard, monitor, headphones
Enter order ID: 3
Enter customer name: dd
Enter items (comma-separated): ww
Log: Processing order #3 for dd. Items: ww
Log: error: ItemNotAvailable "ww"
Result: failed: ItemNotAvailable "ww"
```

Рис. 2. Заказ с несуществующим товаром

```
Available items: laptop, phone, mouse, keyboard, monitor, headphones
Enter order ID: 3
Enter customer name: ss
Enter items (comma-separated):
Log: Processing order #3 for ss. Items:
Log: error: EmptyOrder
Result: failed: EmptyOrder
```

Рис. 3. Заказ с пустым списком товаров

Тестирование подтвердило, что программа работает корректно, обрабатывая разные ошибки и логируя все действия.

Выводы

В ходе работы релизована программа, реализующая систему обработки заказов с логированием и обработкой ошибок. Определены типы данных заказа `Order`, склада `Inventory` и возможных ошибок `OrderError`. Написана функция логирования информации о начала обработки заказа `logMessage`, успешной и ошибочной обработке `logSuccess` и `logError`. Написана функция проверки заказа `validateOrder` и главная функция `main`. Тестирование программы показало, что все функции работают корректно и обрабатывают частные случаи.

Список литературы

- [1] Курт У. «Программируй на Haskell». М.: Издательство, 2019. 320 стр.
- [2] Липовача М. «Изучай Haskell во имя добра». М.: ДМК Пресс, 2012. 490 стр.

Приложение 1 «Исходный код программы»

Листинг 11. Код работы

```
1 {-# LANGUAGE OverloadedStrings #-}
2
3 module Inventory (
4   Item,
5   OrderId,
6   CustomerName,
7   Order(..),
8   Inventory,
9   OrderError(..),
10  OrderProcessor,
11  validateOrder,
12  logMessage,
13  logSuccess,
14  logError
15 ) where
16
17 import Control.Monad.Except
18 import Control.Monad.Writer
19 import Control.Monad (when)
20 import Data.List (intercalate)
21
22 -- Order - информация о заказе, поля: уникальный идентификатор заказа, имя
    покупателя, список товаров
23 type Item = String
24 type OrderId = Int
25 type CustomerName = String
26
27 data Order = Order{
28   orderId :: OrderId
29   , customerName :: CustomerName
30   , items :: [Item]
31 } deriving (Show, Eq)
32
33 -- Inventory - текущий склад магазина: список доступных товаров
34 type Inventory = [Item]
35
36 -- OrderError - возможные ошибки при обработке заказа: товара нет в наличии,
    пустой список товаров некорректный( заказ)
37 data OrderError
38 = EmptyOrder
39 | ItemNotAvailable Item
40 deriving (Show, Eq)
41
42 -- Псевдоним типа для трансформера монад
43 type OrderProcessor = ExceptT OrderError (Writer [String])
44
45 -- Функции логирования
46 writeLog :: String -> OrderProcessor ()
47 writeLog message = tell [message]
48
49 logMessage :: Order -> OrderProcessor ()
50 logMessage order = do
```

```

51 writeLog $ "Log: Processing order #" ++ show (orderId order) ++ " for "
    ++ customerName order ++ ". Items: " ++ intercalate ", " (items order
    )
52
53 logSuccess :: Order -> OrderProcessor ()
54 logSuccess order =
55 writeLog $ "Log: Order #" ++ show (orderId order) ++ " successfully
    processed!"
56
57 logError :: OrderError -> OrderProcessor ()
58 logError error =
59 writeLog $ "Log: error: " ++ show error
60
61 -- Функция проверки заказа
62 validateOrder :: Inventory -> Order -> OrderProcessor Order
63 validateOrder inventory order = do
64 logMessage order
65 when (null (items order)) $ do
66 logError EmptyOrder
67 throwError EmptyOrder
68 let checkItem item =
69 if item `elem` inventory
70 then return ()
71 else do
72 logError (ItemNotAvailable item)
73 throwError (ItemNotAvailable item)
74 mapM_ checkItem (items order)
75 logSuccess order
76 return order
77
78 module Main where
79
80 import Inventory
81 import Control.Monad.Except (runExceptT)
82 import Control.Monad.Writer (runWriter)
83 import Data.List (intercalate)
84 import System.IO
85 import Text.Read (readMaybe)
86
87 main :: IO ()
88 main = do
89 let testInventory = ["laptop", "phone", "mouse", "keyboard", "monitor",
    "headphones"]
90 putStrLn ""
91 putStrLn $ "Available items: " ++ intercalate ", " testInventory
92 orderId <- readOrderId
93 customerName <- readCustomerName
94 itemsList <- readItems
95 let order = Order orderId customerName itemsList
96 let (result, logMessages) = runWriter (runExceptT (validateOrder
    testInventory order))
97 mapM_ putStrLn logMessages
98 case result of
99 Left error -> putStrLn $ "Result: failed: " ++ show error
100 Right order -> putStrLn $ "Result: success! Order processed."
101

```

```

102 readOrderId :: IO OrderId
103 readOrderId = do
104   putStr "Enter order ID: "
105   hFlush stdout
106   input <- getLine
107   case readMaybe input of
108     Just id | id > 0 -> return id
109     Just _ -> do
110       putStrLn "Order ID must be greater than 0"
111       readOrderId
112     Nothing -> do
113       putStrLn "Invalid order ID. Please enter a number."
114       readOrderId
115
116 readCustomerName :: IO CustomerName
117 readCustomerName = do
118   putStr "Enter customer name: "
119   hFlush stdout
120   name <- getLine
121   if null (trim name)
122   then do
123     putStrLn "Customer name cant be empty"
124     readCustomerName
125   else return name
126 where
127   trim = dropWhile (== ' ') . reverse . dropWhile (== ' ') . reverse
128
129 readItems :: IO [Item]
130 readItems = do
131   putStr "Enter items (comma-separated): "
132   hFlush stdout
133   input <- getLine
134   return $ splitOnComma input
135 where
136   splitOnComma [] = []
137   splitOnComma str =
138     let (item, rest) = break (== ',') str
139     in trim item : case rest of
140       [] -> []
141       (_:rest') -> splitOnComma rest'
142   trim = dropWhile (== ' ') . reverse . dropWhile (== ' ') . reverse

```